PureFeed

Technical Documentation

Version: 1.0 (April 2026)

Repository: github.com/JuliaSivridi/PureFeed

Bot: @pure_feed_bot

Stack: Python 3.11 · FastAPI · Telethon · SQLite · Docker

> **What it does:** PureFeed connects to each user's Telegram account via MTProto userbot, monitors subscribed channels in real time, filters out ads by keyword rules, and forwards clean content to private destination channels — complete with unread counters, via a shared relay buffer.

Table of Contents

1. Overview
2. Tech Stack
3. Project Architecture
4. Data Model
5. Multi-User System
6. Monitoring Engine
7. Relay & Unread Counters
8. Filtering Engine
9. Bot Commands UI
10. Auth Flow (QR Login)
11. Setup & Deployment

1. Overview

PureFeed is a multi-user Telegram feed filter delivered as a **self-hosted Docker service**. Each user connects their own Telegram account as a userbot (via Telethon + MTProto), configures independent feeds with source channels and keyword filters, and receives clean forwarded content in their private destination channels — with working unread counters.

Key Design Goals

- **No web UI** — everything managed through Telegram bot commands and inline keyboards
- **Per-user isolation** — each user has their own authenticated session, feeds, and monitoring process
- **Unread counter** — achieved via a shared Relay Buffer channel; the bot (not the userbot) is the sender, so Telegram marks messages as unread
- **Zero-friction auth** — QR code login only; no phone number input, no SMS codes
- **Album support** — media groups (multi-photo posts) forwarded atomically as a group

2. Tech Stack

Layer	Technology	Version	Notes
Language	Python	3.11	asyncio throughout; type hints
Web framework	FastAPI	0.111	Single `/api/status` endpoint; lifespan for startup
ASGI server	Uvicorn	0.29	Runs inside Docker container
MTProto client	Telethon	1.36	One `TelegramClient` per user; SQLite session files
Bot API client	httpx	0.27	Direct HTTPS calls to api.telegram.org; no SDK
Database	SQLite via aiosqlite	0.20	Single `feeds.db` file; async access
Container	Docker + Compose	—	`python:3.11-slim` base image
QR codes	qrcode + Pillow	7.4 / 10.3	Generated as PNG, sent via Bot API `sendPhoto`

3. Project Architecture

```
app/
├── main.py # FastAPI app, lifespan: init_db → start_all → run_polling
├── bot.py # TelegramBot class — one instance per user
│ # TelegramClient (Telethon), _setup_handler, _process_message
│ # _forward_message / _forward_media_group (relay path)
│ # start_auth / confirm_auth, start_qr_auth / wait_for_qr_scan
│ # _init_bot_relay — join channel + promoteChatMember
├── session_manager.py # SessionManager — dict[user_id → TelegramBot]; start_all/stop_all
├── bot_commands.py # Bot API long-poll loop; all screens and conversation states
│ # run_polling, handle_message, handle_callback
│ # show_feed_list, show_feed, show_settings, show_channels, …
├── database.py # aiosqlite CRUD; init_db with migrations; normalize_channel()
├── filter.py # AdFilter.is_ad() — keyword matching only (no AI)
└── api.py # GET /api/status — active user count

data/ # Docker volume mount (./data:/app/data)
├── feeds.db # SQLite database
└── telegram_*.session # Telethon session file per user (telegram_{user_id}.session)
```

Startup Sequence

1. `main.py` lifespan runs: `init_db()` creates/migrates SQLite tables
2. `session_manager.start_all()` loads all rows from `user_settings` and calls `TelegramBot.start_monitoring()` for each
3. `run_polling(session_manager)` starts as an asyncio task — long-polls Bot API `getUpdates` (30 s timeout) indefinitely
4. FastAPI/Uvicorn starts serving HTTP on port 8000

Message Flow

1. Telethon `NewMessage` event fires for a subscribed channel
2. `_process_message()` matches `chat.id` against `id_to_feeds` (pre-resolved numeric IDs)
3. Matched feeds iterated; `ad_filter.is_ad()` called per feed
4. Clean messages: **relay path** — userbot forwards to Relay Buffer → Bot API `forwardMessage` → destination
5. Source message marked as read via `client.send_read_acknowledge()`

4. Data Model

All data lives in a single SQLite file `/app/data/feeds.db` with five tables.

feeds

Column	Type	Notes
id	INTEGER PK	Auto-increment
name	TEXT	User-defined feed name
destination_channel	TEXT	Numeric ID (`-100XXXXXXXXXX`) or `@username`
use_ai_filter	INTEGER	Reserved; always 0 (AI filtering removed)
enabled	INTEGER	0 / 1 — paused feeds are skipped by the monitoring handler
user_id	INTEGER	Telegram user ID of the feed owner; isolates feeds per user
created_at	TEXT	ISO 8601 UTC

source_channels

Column	Type	Notes
id	INTEGER PK	
feed_id	INTEGER FK	→ feeds.id ON DELETE CASCADE
channel_username	TEXT	Stored as normalized key: `@username` or `-100XXXXXXXXXX`

filter_keywords

Column	Type	Notes
id	INTEGER PK	
feed_id	INTEGER FK	→ feeds.id ON DELETE CASCADE
keyword	TEXT	Stored lowercase; supports `word+word` AND-rules

user_settings

Column	Type	Notes

user_id	INTEGER PK	Telegram user ID
api_id	INTEGER	From my.telegram.org
api_hash	TEXT	From my.telegram.org

channel_last_seen

Column	Type	Notes
channel_username	TEXT PK	
last_message_id	INTEGER	Used to avoid reprocessing old messages after restart
updated_at	TEXT	ISO 8601 UTC

Channel Normalization

`normalize_channel()` in `database.py` converts any input format to a canonical key before storage:

Input	Stored as
`https://t.me/channelname`	`@channelname`
`channelname`	`@channelname`
`https://t.me/c/1812695632/5`	`-1001812695632`
`https://t.me/+AbCdEfGhIjK`	Kept as-is (invite link)

5. Multi-User System

Each user who authenticates gets their own `TelegramBot` instance, independent session file, and independent channel handler. `SessionManager` is the single point of coordination.

```
class SessionManager:
    _bots: dict[int, TelegramBot]   # user_id → TelegramBot

    async def start_all()           # called on app startup
    async def get_or_create(user_id, api_id, api_hash) -> TelegramBot
    def get(user_id) -> TelegramBot | None
    async def stop_all()            # called on shutdown
```

Per-User Isolation

- **Session file:** `/app/data/telegram_{user_id}.session` — separate Telethon SQLite per user
- **Feeds:** `get_all_feeds(user_id=...)` filters by `feeds.user_id`; the bot UI only shows the caller's feeds
- **Handler:** each `TelegramBot` registers its own `NewMessage` handler on its own `TelegramClient`
- **Relay:** the shared Relay Buffer channel is used by all userbots; each userbot is auto-promoted to admin on first `start_monitoring()`

6. Monitoring Engine

Two-Phase Handler Setup

On `start_monitoring()`, the handler is registered twice to handle messages that arrive during the slow `iter_dialogs()` call:

1. **Phase 1** — `_setup_handler(resolve_ids=False)`: registers a global `NewMessage` handler immediately. Channel IDs cannot be resolved yet (entity cache is empty), so only string-key fallback matching works.
2. **Dialogs load** — `iter_dialogs()` populates Telethon's entity cache (~1–2 s for hundreds of dialogs).
3. **Phase 2** — `_setup_handler(resolve_ids=True)`: rebuilds the handler. All `@username` keys are resolved to numeric IDs and stored in `id_to_feeds`. The old handler is removed first.

Channel Matching

```
# id_to_feeds: dict[int, list[dict]]  — numeric_channel_id → feeds
# Built in _setup_handler(); used in _process_message()

if chat.id in id_to_feeds:
    feeds_for_channel = id_to_feeds[chat.id]      # fast path (resolved)
else:
    # fallback: string comparison by @username or "-100{id}"
```

Keepalive Loop

Telegram does not always push `NewMessage` events for channels promptly. Every 30 seconds, `_updates_keepalive()` calls `GetStateRequest()` which forces Telethon to fetch missed updates. Without this, delay can reach 10–15 minutes.

Media Groups (Albums)

Multi-photo posts carry a `grouped_id`. A 1-second buffer collects all parts, then forwards the whole album atomically via Bot API `forwardMessages` (plural). The timer resets on each new part, so late-arriving photos are always included.

```
_media_groups:       dict[grouped_id, {messages, source_name, feeds}]
_media_group_tasks:  dict[grouped_id, asyncio.Task]

# Each new photo: cancel existing task → create new task with 1 s sleep
# After sleep: pop group → filter → forwardMessages
```

7. Relay & Unread Counters

When a bot token sends a message to a channel where the user is a member, Telegram marks it as unread and increments the badge counter. Direct userbot forwards do not trigger this. The Relay Buffer pattern solves this:

> **Relay path (single message):**
> Userbot → `forward_messages(relay_entity, msg, silent=True)` →
> Bot API `forwardMessage(chat_id=destination, from_chat_id=relay, message_id=relay_id)` →
> Userbot `delete_messages(relay_entity, [relay_id])`

> **Relay path (album):**
> Userbot → `forward_messages(relay_entity, messages, silent=True)` →
> Bot API `forwardMessages(chat_id=destination, from_chat_id=relay, message_ids=[…])` →
> Userbot deletes all relay copies

Relay Channel Requirements

- Private channel, created once by the service owner
- **Bot token** must be admin with "Add Admins" permission
- Each user's userbot is auto-joined via `RELAY_INVITE_LINK` and auto-promoted to admin via Bot API `promoteChatMember` on `start_monitoring()`
- If the Bot API call fails with `can't remove chat owner`, it is silently ignored (owner already has all rights)

Fallback

If relay fails for any reason, the message is forwarded directly by the userbot (no unread counter, but delivery is guaranteed).

8. Filtering Engine

`filter.py` exposes a single async method: `ad_filter.is_ad(text, keywords) → (bool, reason)`. Filtering is keyword-only; there is no AI or built-in pattern list.

Keyword Rules

- **Single word:** `реклама` — matches if the word appears anywhere in the message (case-insensitive)
- **AND rule:** `скидка+купить` — all parts separated by `+` must be present simultaneously
- Keywords stored lowercase; text is lowercased before comparison

```
def _check_keywords(self, text, keywords):
    text_lower = text.lower()
    for rule in keywords:
        parts = [p.strip() for p in rule.split("+") if p.strip()]
        if all(part in text_lower for part in parts):
            return True, f"Найдено правило: '{' + '.join(parts)}'"
    return False, ""
```

> Messages with no text (pure media, stickers, polls) return `(False, "")` immediately — they are never filtered, even if the album has no caption.

9. Bot Commands UI

All user interaction happens via Telegram bot inline keyboards and text messages. The polling loop in `run_polling()` calls `getUpdates` with a 30 s long-poll timeout and dispatches to `handle_message()` or `handle_callback()`.

Conversation State Machine

Per-chat state is stored in `_states: dict[int, {step, ctx}]` (in-memory). State is cleared on any `/` command, and on `/esc` which also navigates back to the appropriate screen.

Step	Triggered by	Expects
`new_feed_name`	➕ Добавить фид	Text → feed name
`rename_feed`	✏️ Название	Text → new name
`set_dest`	📺 Канал назначения	Forwarded message from destination channel
`add_channel`	➕ Добавить канал	Forwarded message from source channel
`add_keyword`	➕ Добавить фильтр	Text → keyword or `word+word`
`set_api_id`	🔑 Ввести api_id	Digits only
`set_api_hash`	🔑 Ввести api_hash	String ≥ 10 chars
`auth_phone`	📱 Войти по телефону	Phone number with country code
`auth_code`	auto after phone	SMS / app code
`auth_2fa`	SessionPasswordNeededError	2FA password
`auth_qr_wait`	📷 Войти через QR-код	User scans QR in Telegram app (background wait)
`auth_2fa_qr`	SessionPasswordNeededError after QR	2FA password

Screen Navigation

All screens use the **send-and-delete** pattern: the previous bot message is deleted before sending a new one, keeping the conversation always scrolled to the latest message.

Screen	Shows
Feed list	Monitoring status · feed buttons with channel count · Start/Stop · ⚙️ Settings

Feed detail	Destination channel · channel count · filter count · toggle enabled · rename · delete
Channels	Channel list · Add · Delete list
Filters	Keyword list · Add · Delete list
Settings	api_id (plain) · api_hash (masked) · auth status · Enter api_id · Enter api_hash · QR Login · Back

Commands

Command	Action
`/start`, `/feedlist`	Open feed list
`/settings`	Open settings screen
`/help`	Send full setup guide (static text, no screen)
`/esc`	Cancel active input state; navigate back

10. Auth Flow (QR Login)

Telegram blocks login attempts from data-center IPs when using phone+code auth. QR login sidesteps this: the server generates a token, the user approves the login on their phone inside the official Telegram app — the server IP is never part of the authorization step.

QR Login Steps

1. User presses 📷 **Войти через QR-код** in /settings
2. Bot calls `bot.start_qr_auth()` → `client.qr_login()` → returns `tg://login?token=…` URL
3. Bot generates a QR code PNG via `qrcode` library, sends it as `sendPhoto`
4. A background asyncio task (`_qr_auth_waiter`) awaits `qr.wait(timeout=120)`
5. User opens Telegram → Settings → Devices → Scan QR code
6. On success: `start_monitoring()` is called; user sees ✅ *Авторизация успешна!* and returns to feed list
7. On `SessionPasswordNeededError`: state transitions to `auth_2fa_qr`; user enters 2FA password
8. On timeout (2 min): user is prompted to retry

> The `/esc` command cancels the QR wait by calling `task.cancel()` stored in the state context, preventing zombie background tasks.

Session Persistence

After successful auth, Telethon writes the session to `/app/data/telegram_{user_id}.session`. On the next `start_all()` (container restart), the session is reused automatically — no re-auth needed.

11. Setup & Deployment

Prerequisites

- VPS / server with Docker and Docker Compose
- Telegram bot token from @BotFather
- A private **Relay Buffer** channel with the bot as admin (with "Add Admins" right)

Environment Variables

Variable	Required	Description
`TELEGRAM_BOT_TOKEN`	✅	Bot token from @BotFather
`RELAY_CHANNEL_ID`	✅	Numeric ID of relay channel, e.g. `-1001234567890`
`RELAY_INVITE_LINK`	✅	Permanent invite link for relay channel (`https://t.me/+…`)

Deployment

```
git clone git@github.com:JuliaSivridi/PureFeed.git
cd PureFeed
cp .env.example .env
nano .env              # fill in the three variables above
mkdir -p data
chmod 777 data/        # required for Docker volume write access on Linux
docker compose up -d
docker compose logs -f
```

Data Directory

`./data/` is mounted into the container as `/app/data/`. It must be writable by the container process (root). Do not delete it — it contains `feeds.db` (all user data) and `*.session` files (auth sessions).

Useful Commands

```
docker compose logs -f        # stream logs
docker compose restart        # restart without rebuild
docker compose up -d --build  # rebuild image after code changes
docker compose down           # stop and remove containers
```

Health Check

Docker Compose has a built-in health check: `GET /api/status` every 30 s. The endpoint returns `{"status": "ok", "active_users": N}` where N is the number of currently running monitoring instances.